

Analisi e Progettazione del Software

Iterativo, evolutivo e agile

Oliviero Riganelli

Processi per lo sviluppo del software

Un **processo per lo sviluppo del software** (o **processo software**) definisce un approccio disciplinato per la costruzione, il rilascio e la manutenzione del software

- definisce **chi** fa che **cosa**, **quando** e **come** per raggiungere un certo obiettivo
- “**cosa**” sono le **attività** (es. Individuare attori e casi d’uso) “**chi**” sono i **ruoli** (es. Analista di Sistema), “**come**” sono le **metodologie**, “**quando**” riguarda l’**organizzazione temporale** delle attività

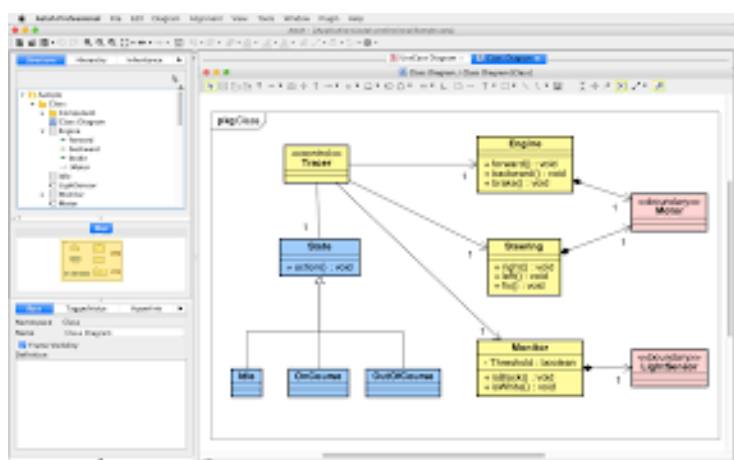
Attività fondamentali di processo



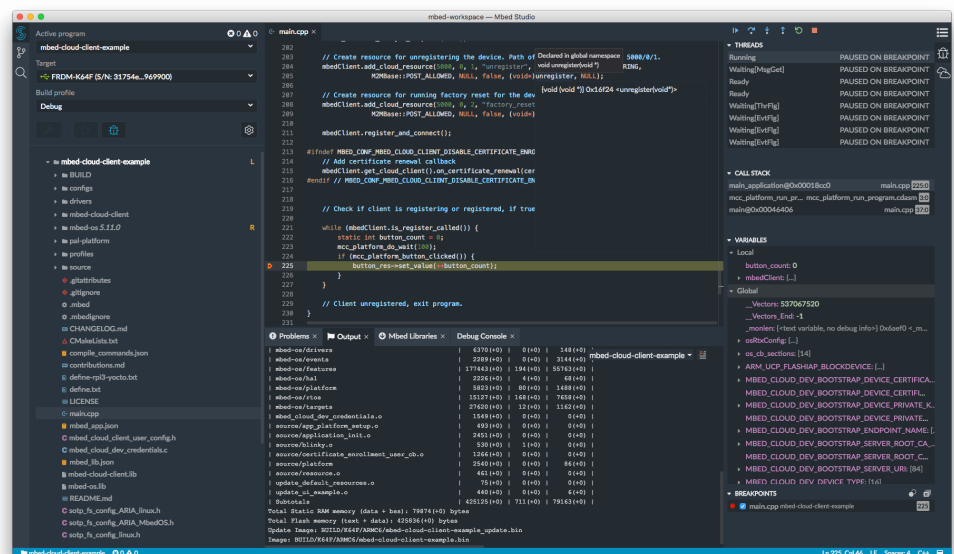
Requisiti



Analisi



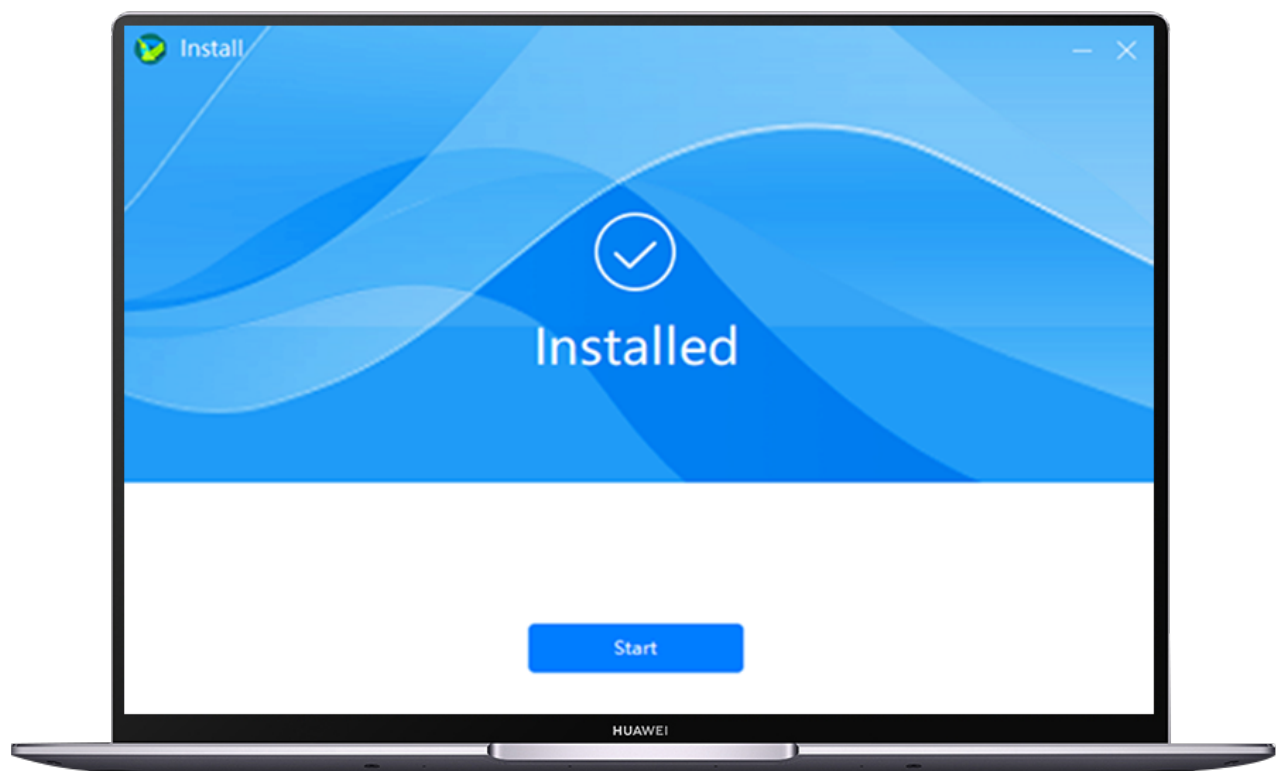
Progettazione



Implementazione



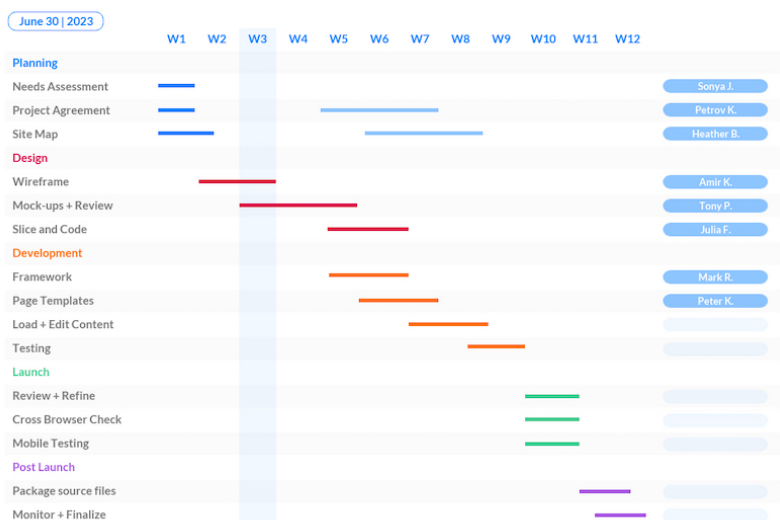
validazione



Rilascio e installazione

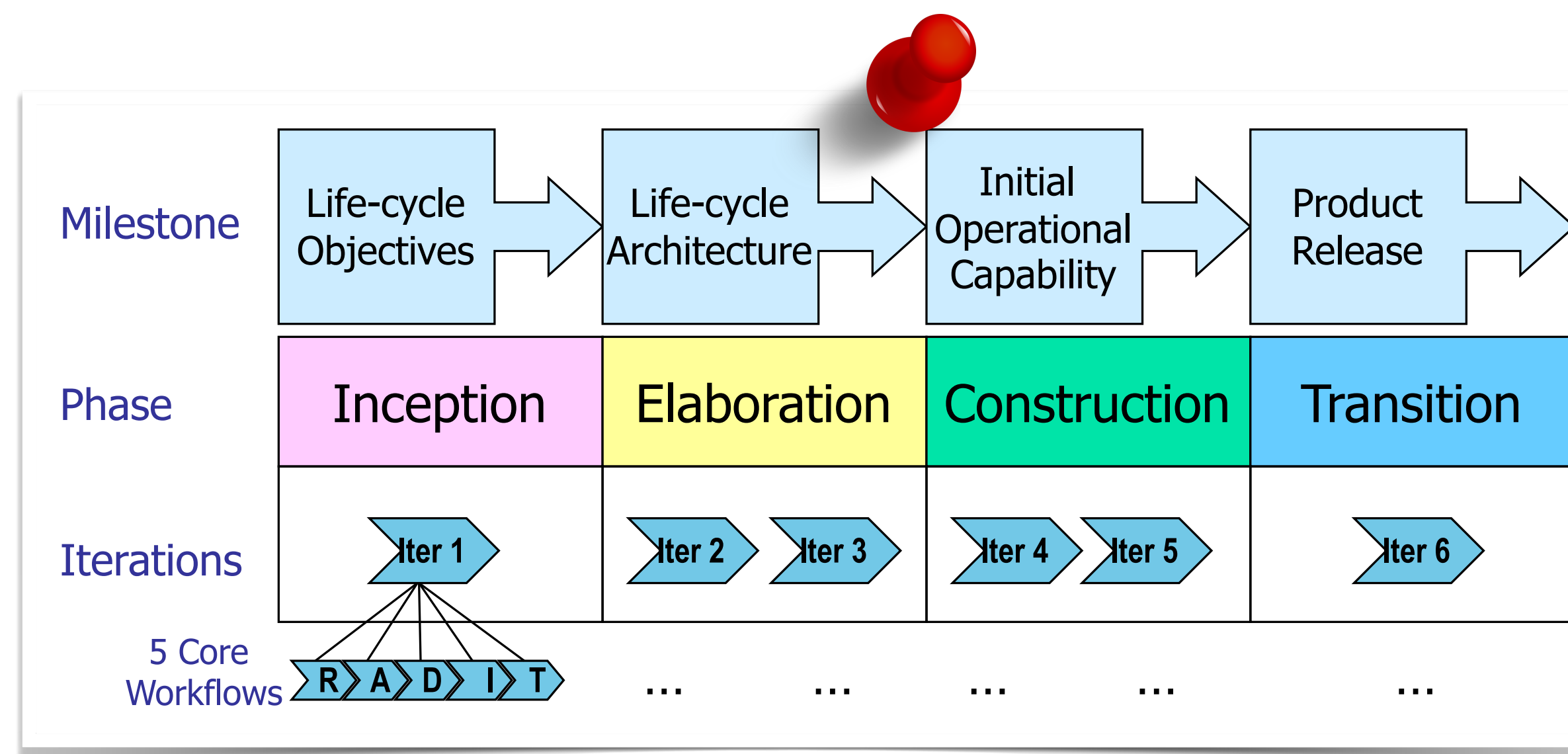
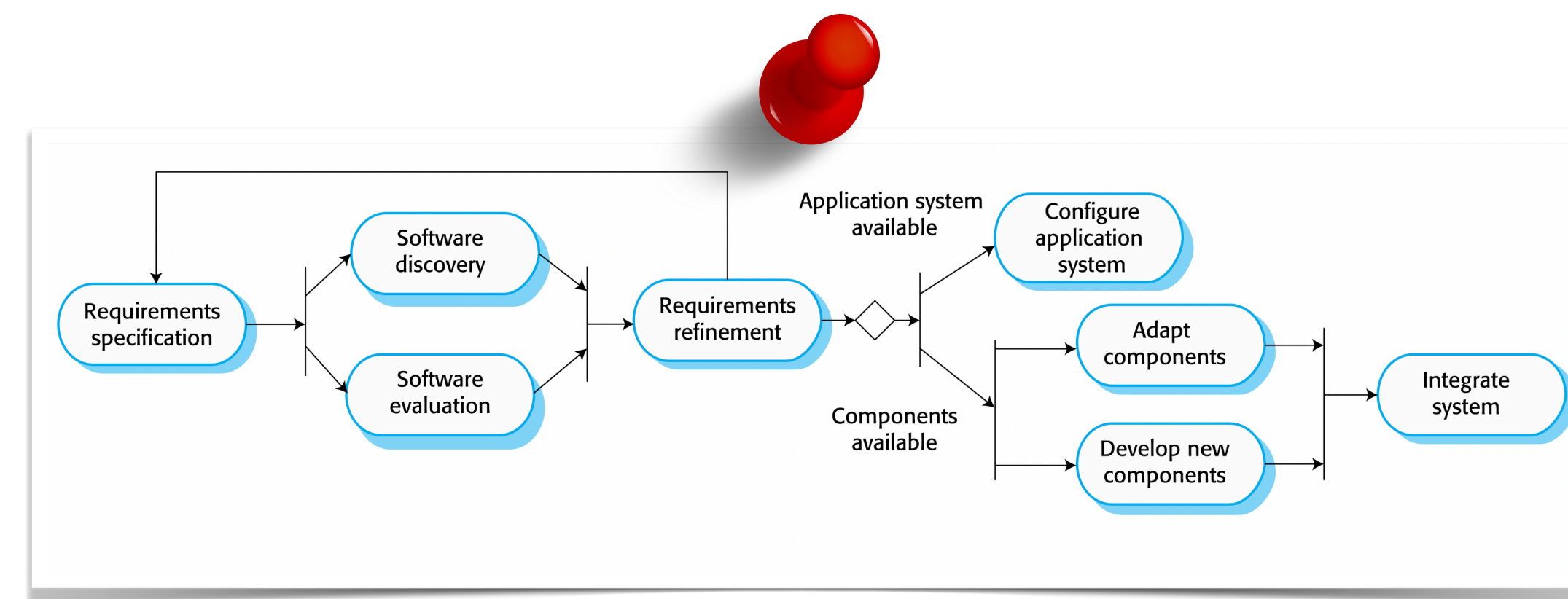
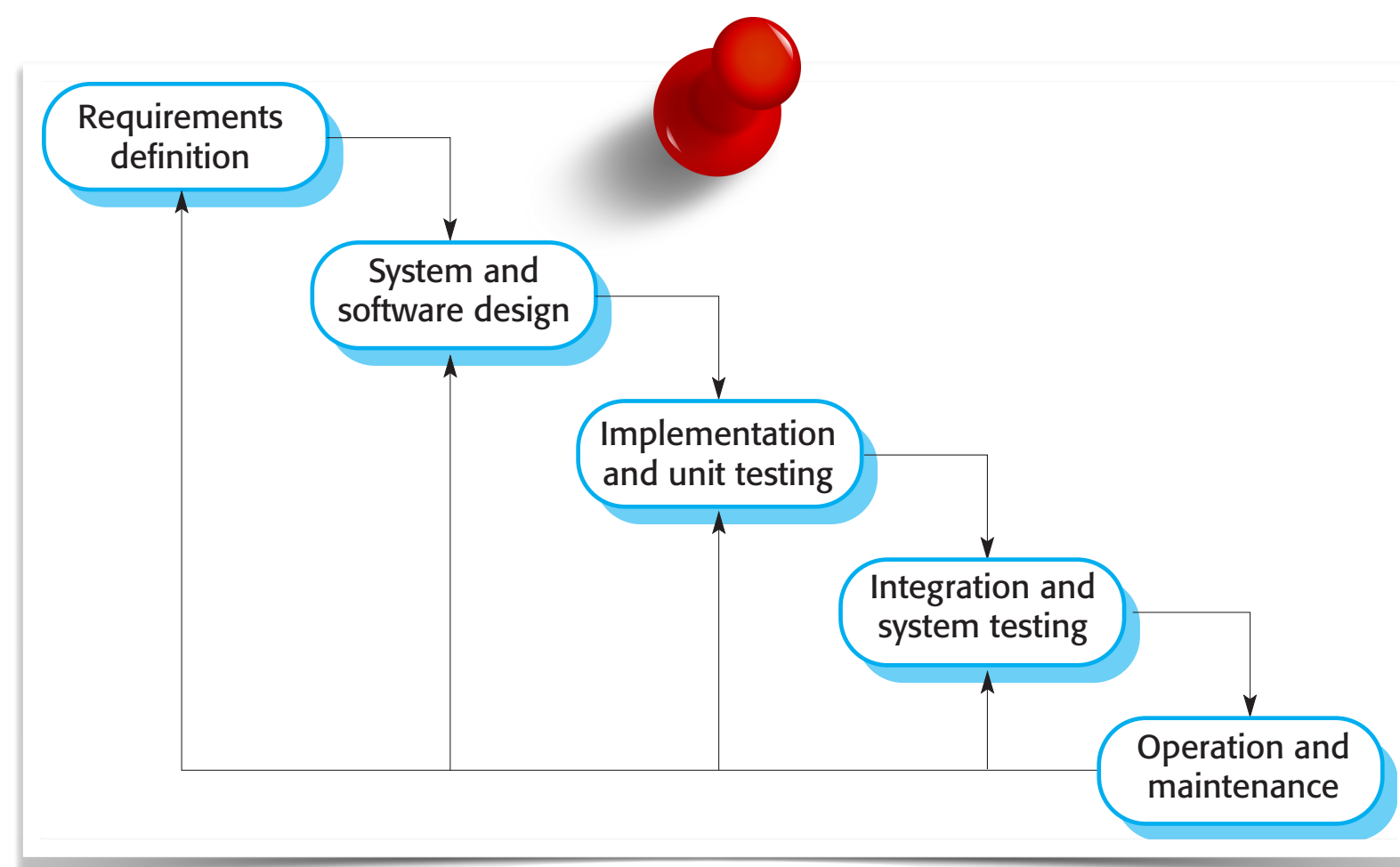


Manutenzione
Ed Evoluzione



Gestione
del Progetto

Perché esistono diversi processi software?



Processi agili e basati sul piano

- I processi orientati al piano sono processi in cui tutte le attività del processo sono pianificate in anticipo e i progressi sono misurati rispetto a questo piano.
- Nei processi agili, la pianificazione è incrementale ed è più facile modificare il processo per riflettere le mutevoli esigenze dei clienti.
- In pratica, la maggior parte dei processi pratici include elementi di entrambi gli approcci, quello guidato dal piano e quello agile.
- Non esistono processi software giusti o sbagliati.

Modelli di processo software

- Il modello a cascata
 - *Modello guidato dal piano. Fasi separate e distinte di specifica e sviluppo.*
- Sviluppo incrementale
 - *Specifica, sviluppo e validazione si alternano. Può essere guidato dal piano o agile.*
- Integrazione e configurazione
 - *Il sistema viene assemblato a partire da componenti configurabili esistenti. Può essere pianificato o agile.*
- In pratica, la maggior parte dei sistemi di grandi dimensioni viene sviluppata utilizzando un processo che incorpora elementi di tutti questi modelli.

Quale Scegliere?

Ecco alcuni dei motivi per cui esistono diversi modelli di processi di sviluppo software:

- **Complessità del progetto:** I progetti software possono variare in termini di complessità, da semplici applicazioni a sistemi complessi e mission-critical. La complessità del progetto influenza il livello di formalità e struttura necessari nel processo di sviluppo.
- **Dimensione del team:** Il numero di persone che lavorano a un progetto software influisce sul modo in cui il lavoro viene organizzato e coordinato. I team di grandi dimensioni hanno bisogno di un processo più formale rispetto ai team di piccole dimensioni.
- **Budget e tempistiche:** Il budget e le tempistiche di un progetto software influenzano il modo in cui il progetto viene pianificato e gestito. I progetti con budget e tempistiche limitati hanno bisogno di un processo più snello rispetto ai progetti con budget e tempistiche flessibili.
- **Rischio:** Il rischio associato a un progetto software influenza il livello di rigore e controllo necessari nel processo di sviluppo. I progetti ad alto rischio hanno bisogno di un processo più rigoroso rispetto ai progetti a basso rischio.

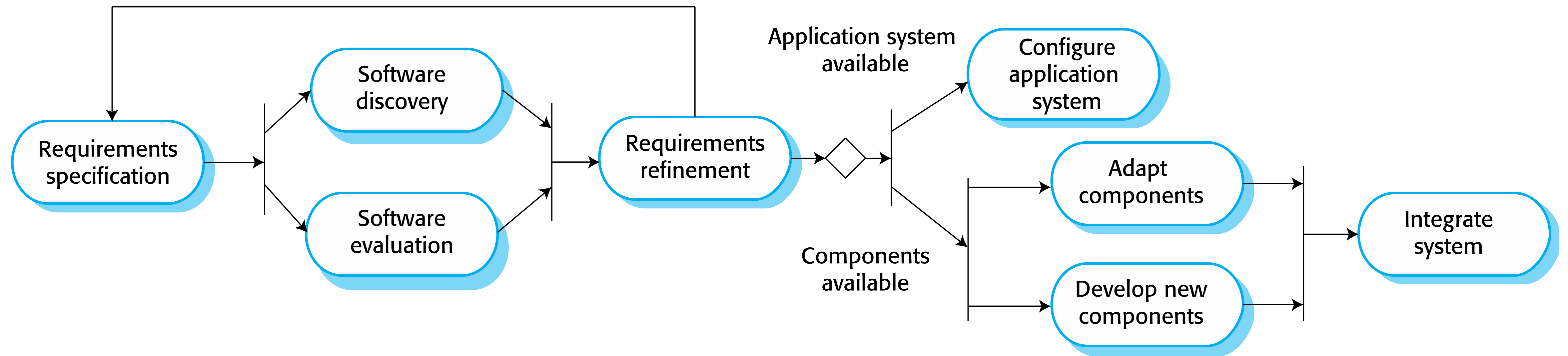
Integrazione e configurazione

- Basato sul riutilizzo del software, in cui i sistemi sono integrati da componenti o sistemi applicativi esistenti (talvolta chiamati sistemi COTS - Commercial-off-the-shelf).
- Gli elementi riutilizzati possono essere configurati per adattare il loro comportamento e la loro funzionalità ai requisiti dell'utente.
- Il riutilizzo è oggi l'approccio standard per la costruzione di molti tipi di sistemi aziendali.

Tipi di software riutilizzabile

- Sistemi applicativi stand-alone (talvolta chiamati COTS) configurati per l'uso in un particolare ambiente.
- Collezioni di oggetti sviluppate come pacchetti da integrare con un framework di componenti come .NET o J2EE.
- Servizi Web sviluppati secondo gli standard di servizio e disponibili per l'invocazione remota.

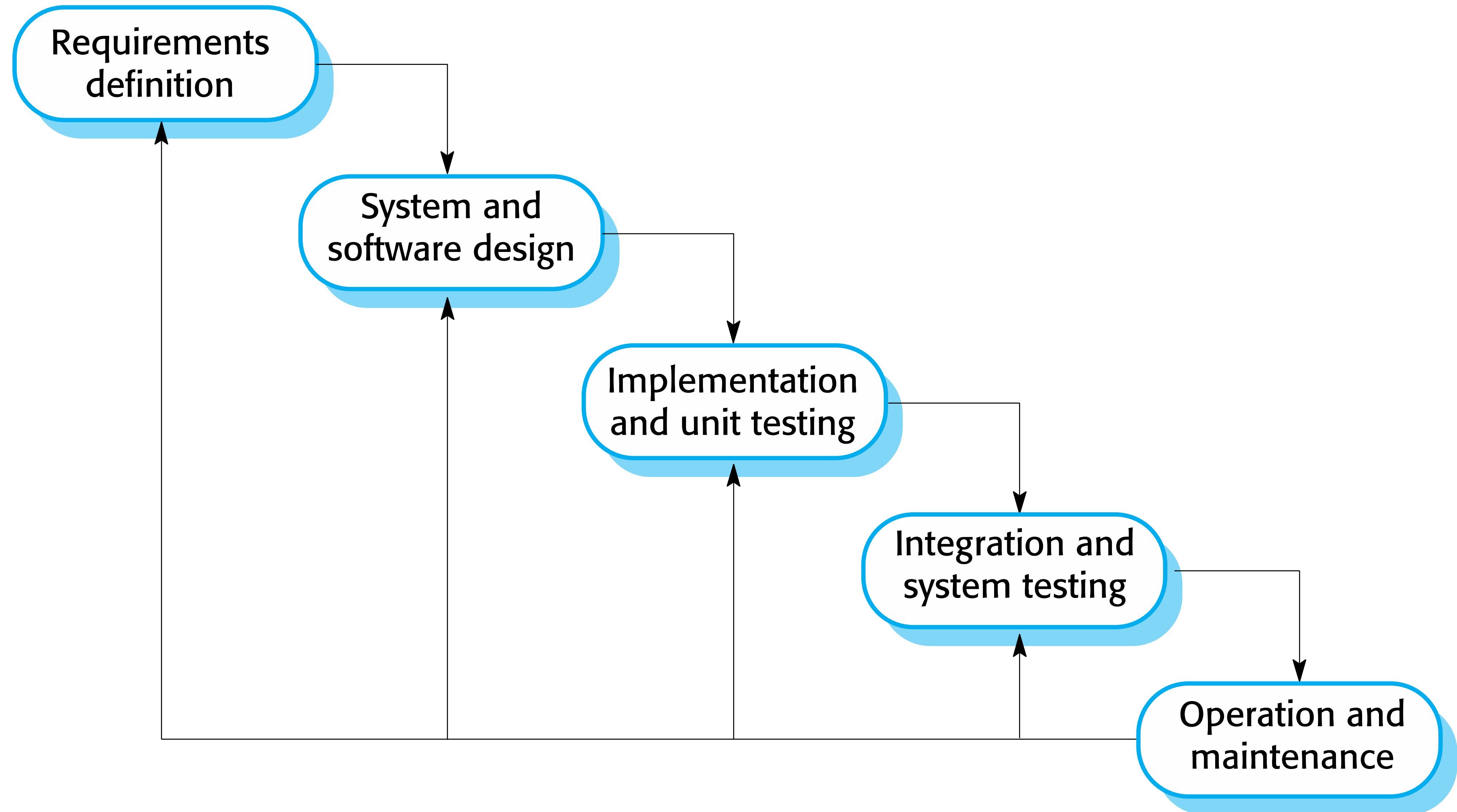
Ingegneria del software orientata al riuso



Vantaggi e svantaggi

- Riduzione dei costi e dei rischi, in quanto viene sviluppato meno software da zero
- Consegna e distribuzione più rapida del sistema
- Ma i compromessi sui requisiti sono inevitabili, quindi il sistema potrebbe non soddisfare le reali esigenze degli utenti
- Perdita di controllo sull'evoluzione degli elementi di sistema riutilizzati

Il processo a cascata



Fasi del modello a cascata

- Il modello a cascata prevede fasi distinte:
 - *Analisi e definizione dei requisiti*
 - *Progettazione del sistema e del software*
 - *Implementazione e test delle unità*
 - *Integrazione e test di sistema*
 - *Funzionamento e manutenzione*
- Il principale svantaggio del modello a cascata è la difficoltà di accogliere i cambiamenti dopo che il processo è stato avviato. In linea di principio, una fase deve essere completata prima di passare alla fase successiva.

Perché il processo a cascata è soggetto a frequenti fallimenti

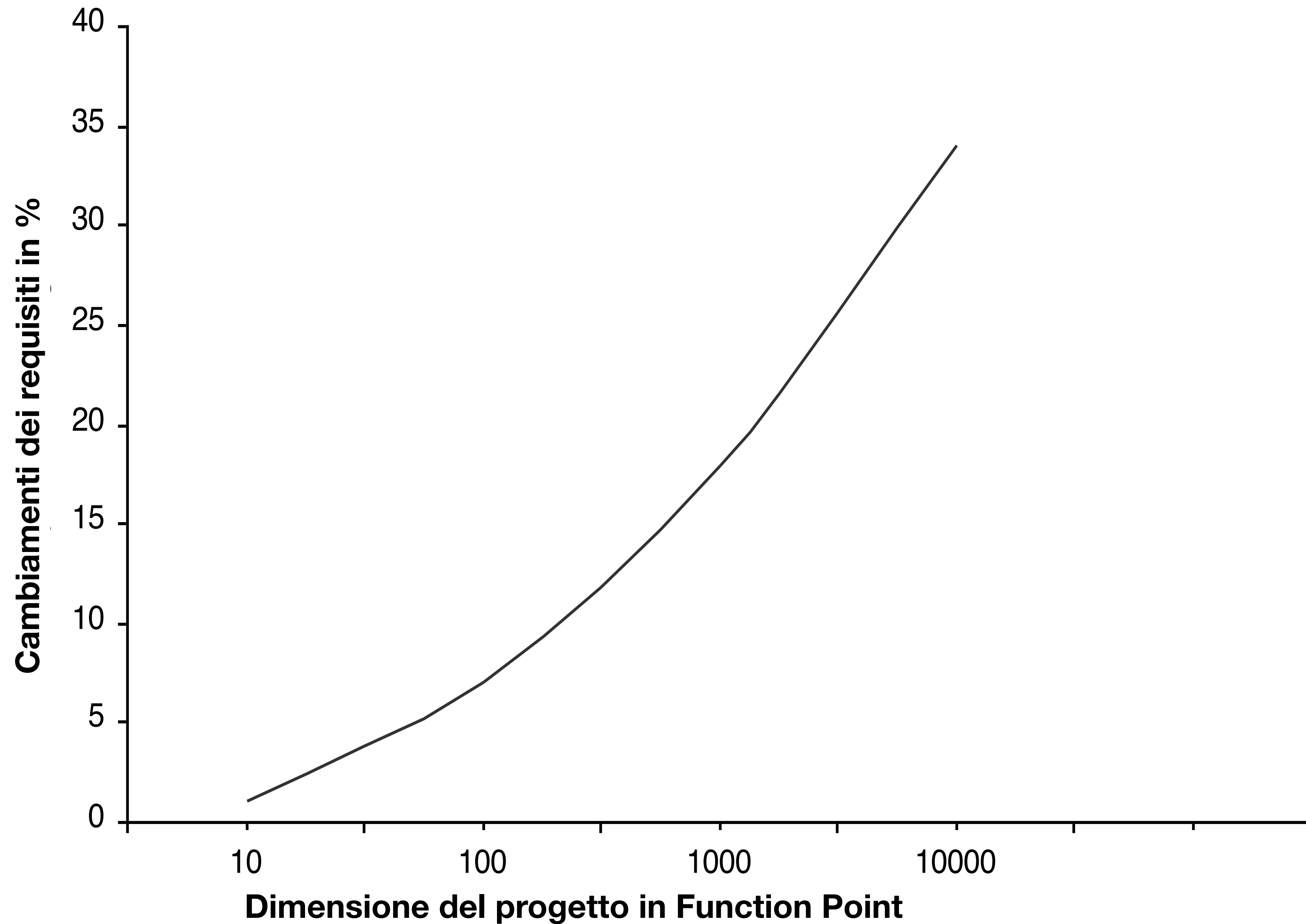
La suddivisione inflessibile del progetto in fasi distinte rende difficile rispondere alle mutevoli esigenze del cliente.

- *Pertanto, questo modello è appropriato solo quando i requisiti sono ben compresi e le modifiche saranno piuttosto limitate durante il processo di progettazione.*
- *Pochi sistemi aziendali hanno requisiti stabili.*

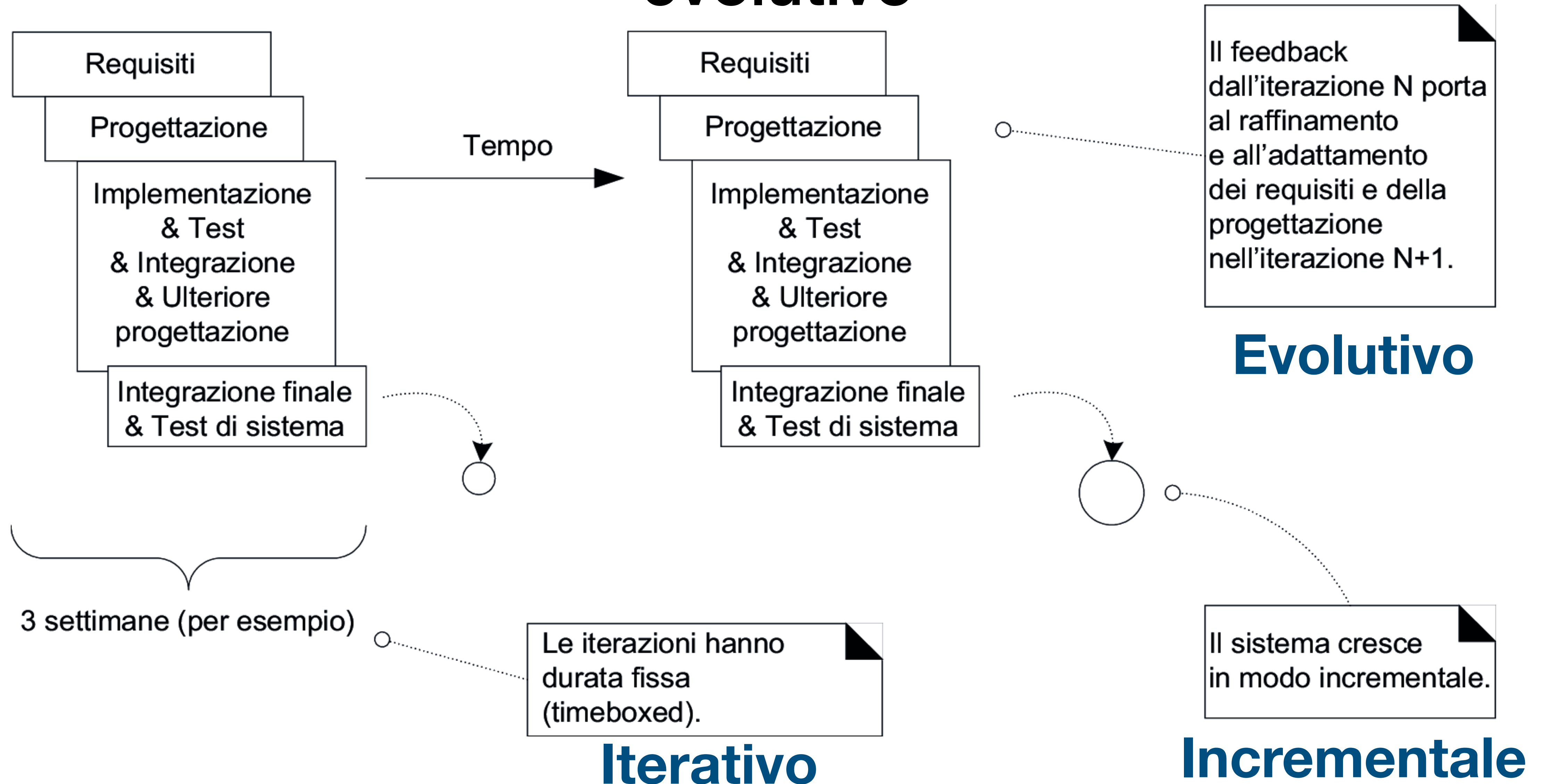
Il modello a cascata è utilizzato soprattutto per progetti di ingegneria dei sistemi di grandi dimensioni, in cui un sistema viene sviluppato in diversi siti.

- *In queste circostanze, la natura pianificata del modello a cascata aiuta a coordinare il lavoro.*

Percentuale di cambiamento in progetti software



Che cosa è lo sviluppo iterativo, incrementale e evolutivo



Come gestire i cambiamenti in un progetto iterativo

- Ciascuna iterazione comporta la scelta di un piccolo sottinsieme di requisiti, una rapida progettazione, implementazione e test.
- il lavoro procede mediante una serie di cicli strutturati di costruzione-feedback-adattamento
- Nel tempo, attraverso il feedback iterativo e l'adattamento, il sistema sviluppato evolve e converge verso i requisiti corretti e il progetto più appropriato.
 - *L'instabilità dei requisiti e del progetto tende a diminuire nel tempo.*
- Nelle iterazioni finali è difficile, ma non impossibile, che si verifichi un cambiamento significativo dei requisiti.

Vantaggi dello sviluppo iterativo

- Minor probabilità di fallimento del progetto, miglior produttività, percentuali più basse di difetti
- Riduzione precoce anziché tardiva dei rischi maggiori
- Progresso visibile fin dall'inizio
- Feedback precoce, coinvolgimento dell'utente e adattamento
- Gestione della complessità

Svantaggi dello sviluppo iterativo

- Il processo non è visibile.
 - *I manager hanno bisogno di documenti regolari per misurare i progressi. Se i sistemi vengono sviluppati rapidamente, non è conveniente produrre documenti che riflettano ogni versione del sistema.*
- La struttura del sistema tende a degradarsi con l'aggiunta di nuovi incrementi.
 - A meno che non si dedichino tempo e denaro al refactoring per migliorare il software, i cambiamenti regolari tendono a corrompere la sua struttura. Incorporare ulteriori modifiche al software diventa sempre più difficile e costoso.

Feedback e adattamento

- feedback proveniente dalle attività di sviluppo
- feedback proveniente dai test e dagli sviluppatori che raffinano il progetto e i modelli
- feedback circa l'avanzamento del team nell'affrontare i requisiti, per raffinare le stime dei tempi e dei costi
- feedback proveniente dal cliente e dal mercato per assegnare/modificare le priorità ai requisiti da affrontare nell'iterazione successiva.

Lunghezza delle iterazioni e timeboxing

- Una buona pratica dello sviluppo iterativo è il **timeboxing**: le iterazioni hanno una lunghezza fissata.
 - *Molti processi iterativi raccomandano una lunghezza da 2 a 6 settimane*
- Passi piccoli, feedback rapido e adattamento sono le idee centrali dello sviluppo iterativo
 - *Iterazioni più lunghe sono contrarie allo spirito dello sviluppo iterativo e aumentano il rischio del progetto*
 - *iterazioni più corte non consentono di completare del lavoro sufficiente a produrre abbastanza software e ottenere dei feedback significativi*
- La durata di un'iterazione, una volta fissata, non può cambiare – un'iterazioni di lunghezza fissata è detta **timeboxed**

Non permettere al pensiero a cascata di invadere un progetto iterativo

- Un pensiero a cascata ha inficiato un progetto interattivo quando ad esempio:
 - *Si scrive la maggior parte dei requisiti o i casi d'uso prima dello sviluppo*
 - *Si crea in modo dettagliato e completo delle specifiche, dei modelli o il progetto prima di iniziare l'implementazione*
- Un'analisi e una modellazione preliminare e corposa, può essere un motivo chiave del fallimento nell'adozione nello sviluppo iterativo

Sviluppo iterativo e flessibilità del codice e del progetto

L'adozione dello sviluppo iterativo richiede che il software venga realizzato in modo **flessibile** affinché l'impatto dei cambiamenti sia basso o tale che l'impatto dei cambiamenti sul sistema sia basso

- A tal fine il codice sorgente deve essere facilmente **modificabile**.
- Per facilitare la modificabilità il codice sorgente deve anche essere facilmente **comprensibile**.

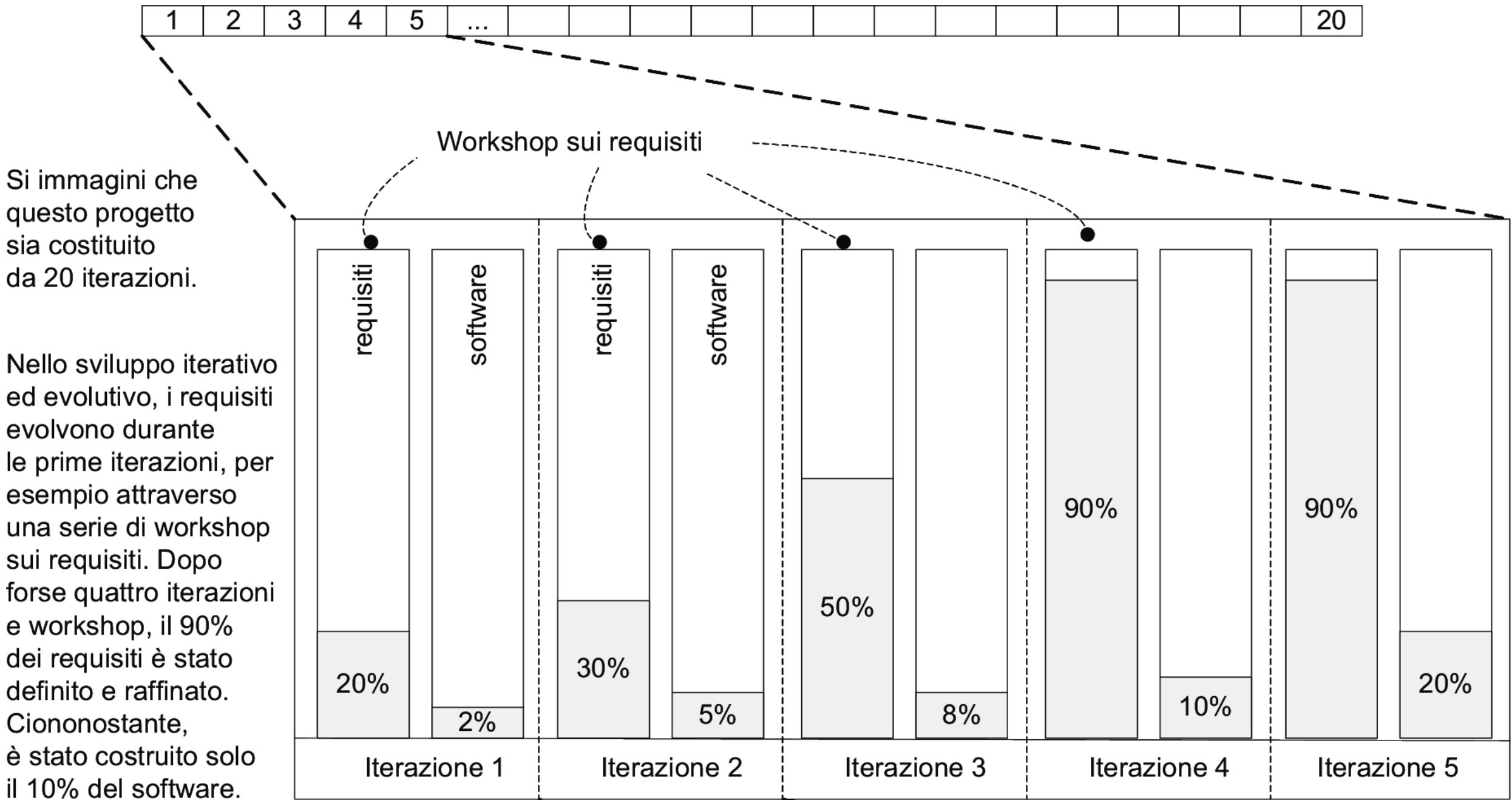
La mancanza di queste qualità rende infatti difficile l'applicazione dello sviluppo iterativo, in cui il codice va esteso in modo incrementale

Come eseguire l’analisi e la progettazione in modo iterativo ed evolutivo

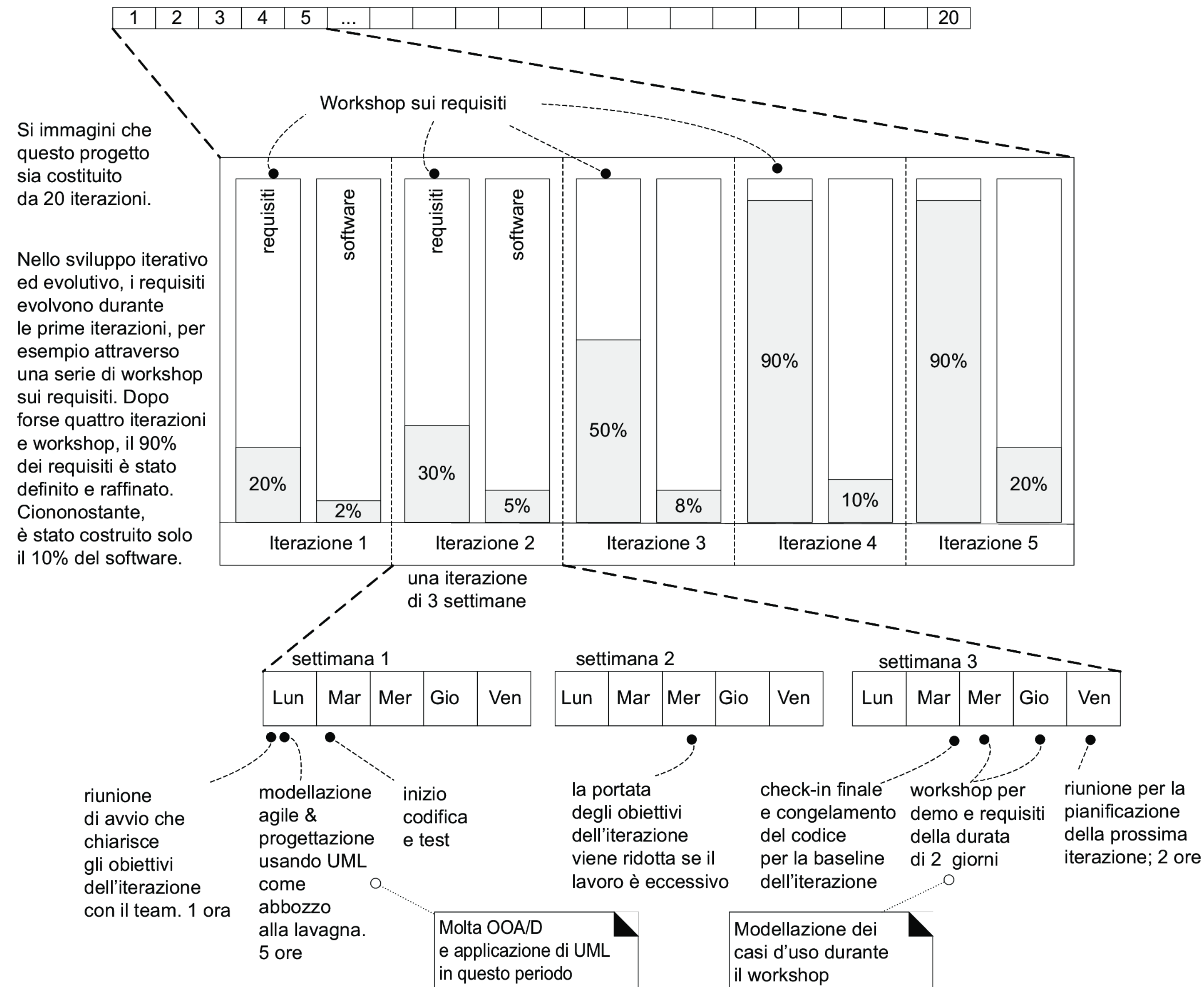
1	2	3	4	5	...													20
---	---	---	---	---	-----	--	--	--	--	--	--	--	--	--	--	--	--	----

Si immagini che
questo progetto
sia costituito
da 20 iterazioni.

Come eseguire l'analisi e la progettazione in modo iterativo ed evolutivo



Come eseguire l'analisi e la progettazione in modo iterativo ed evolutivo



Pianificazione Iterativa

- Un'attività critica nello sviluppo iterativo è la pianificazione delle iterazioni
 - *Non bisogna* tentare di pianificare l'intero progetto in *modo dettagliato* fin dall'inizio
- I processi iterativi promuovono la pianificazione **guidata dal rischio** e **guidata dal cliente**. Gli obiettivi delle iterazioni iniziali vengono scelti:
 - Per identificare ed attenuare i rischi maggiori
 - Per costruire e rendere visibili le caratteristiche a cui il cliente tiene di più
- Lo sviluppo iterativo guidato dal rischio comprende la pratica dello sviluppo iterativo **centrato sull'architettura**
 - Le prime iterazioni si concentrano sulla costruzione il test e la stabilizzazione del nucleo dell'architettura

Non cambiare gli obiettivi dell'iterazione

Durante ciascuna iterazione, i requisiti su cui operare vengono prima fissati (durante la pianificazione iterativa) e poi bloccati (non possono cambiare durante l'iterazione)

- Durante ciascuna iterazione, il team di sviluppo può lavorare al suo meglio
 - *Poiché i requisiti sono bloccati, il team non può essere interrotto o disturbato durante l'iterazione*
 - *I committenti possono interagire con il team di sviluppo a termine di ogni iterazione*

Durante un'iterazione è però possibile che il team di sviluppo decida di cambiare il piano dell'iterazione

- Può avvenire a metà dell'iterazione quando il team valuta la possibilità di raggiungere gli obiettivi dell'iterazione nella sua durata prevista.

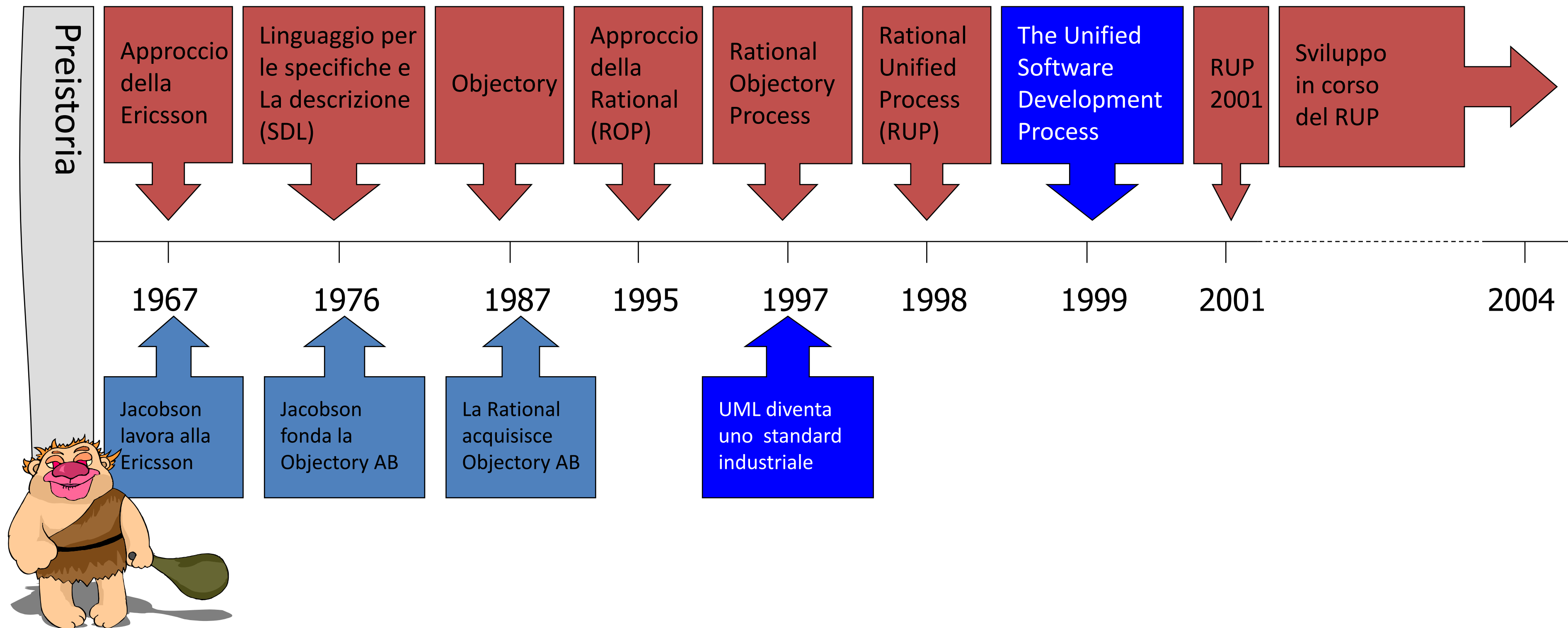
Che cosa è UP

- Il processo unificato (UP) è un processo iterativo diffuso per lo sviluppo del software orientato agli oggetti
- UP è molto flessibile e aperto - incoraggia l'uso di pratiche tratte da altri metodi iterativi (es. Extreme Programming e Scrum)
- UP è
 - *Pilotato dai casi d'uso (requisiti) e dai fattori di rischio*
 - *Incentrato sull'architettura*
 - *Iterativo, incrementale e evolutivo*

Altre pratiche fondamentali di UP

- Affrontare le problematiche di rischio maggiore e valore elevato nelle interazioni iniziali
- Impegnare gli utenti continuamente sulla valutazione, il feedback e i requisiti
- Creare un'architettura coesa nelle interazioni iniziali
- Verificare continuamente la qualità; testare presto, spesso in modo realistico
- Fare della modellazione visuale con UML
- Gestire attentamente requisiti
- Gestire richieste di cambiamento e le configurazioni

UP Storia

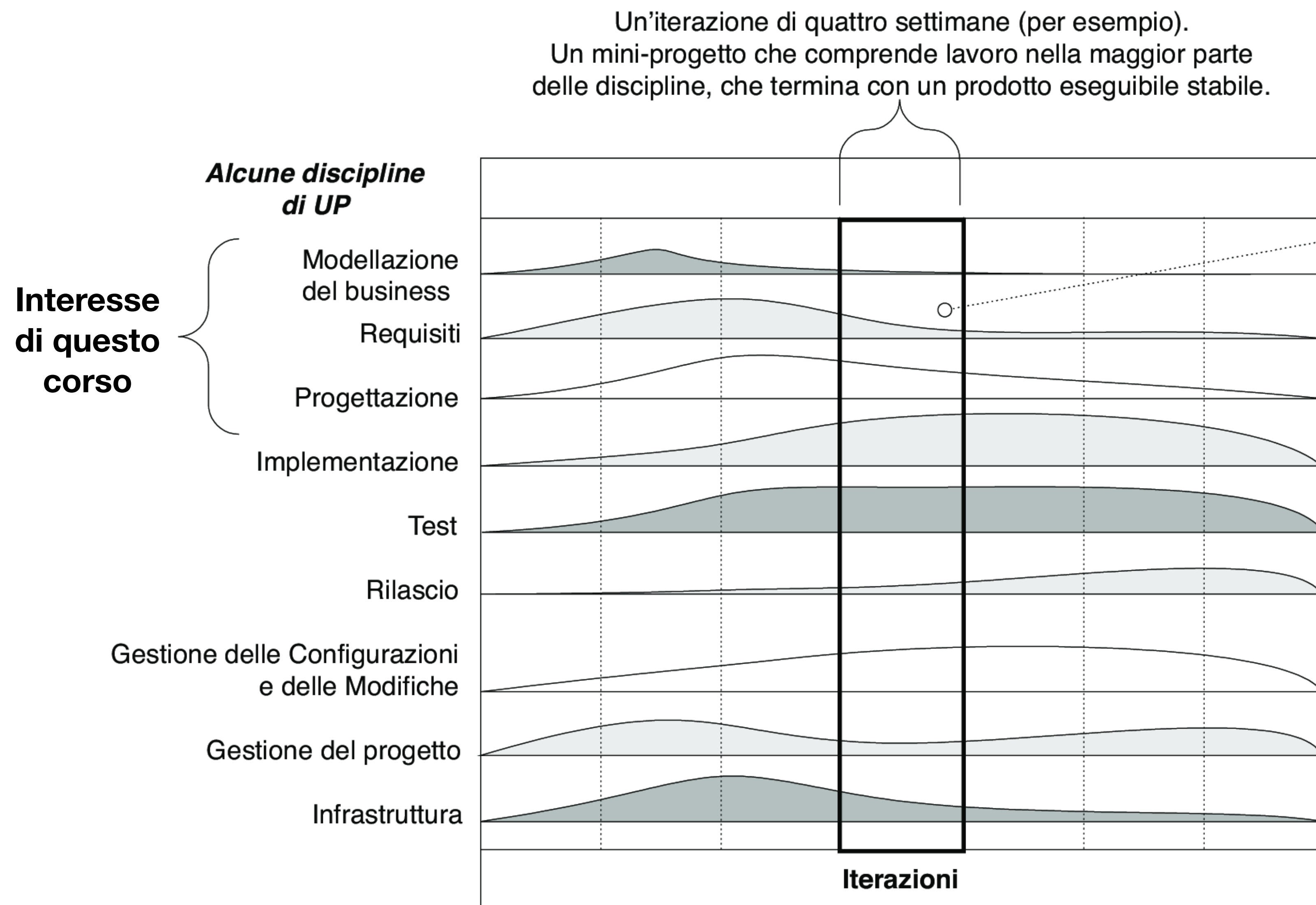


Iterazioni

- Le iterazioni sono la chiave per UP
- Ogni iterazione è come un mini-progetto che include:
 - *Pianificazione*
 - *analisi e progettazione*
 - *costruzione*
 - *integrazione e test*
 - *un rilascio*
- Si arriva al rilascio di un prodotto finale attraverso una sequenza di iterazioni
- Le iterazioni possono sovrapporsi - questo permette lo sviluppo parallelo e il lavoro flessibile in grandi squadre
 - *Richiede un'attenta pianificazione*

Iterazioni e discipline (o flussi di lavoro)

- Anche se ogni iterazione può prevedere tutti i flussi di lavoro, la collocazione dell'iterazione all'interno del ciclo di vita del progetto determina una maggiore enfasi su uno dei flussi di lavoro



Si noti che, nonostante l'iterazione includa lavoro nella maggior parte delle discipline, l'impegno relativo e l'enfasi cambiano nel tempo. Questo esempio è solo un suggerimento, non è da prendere alla lettera.

Release

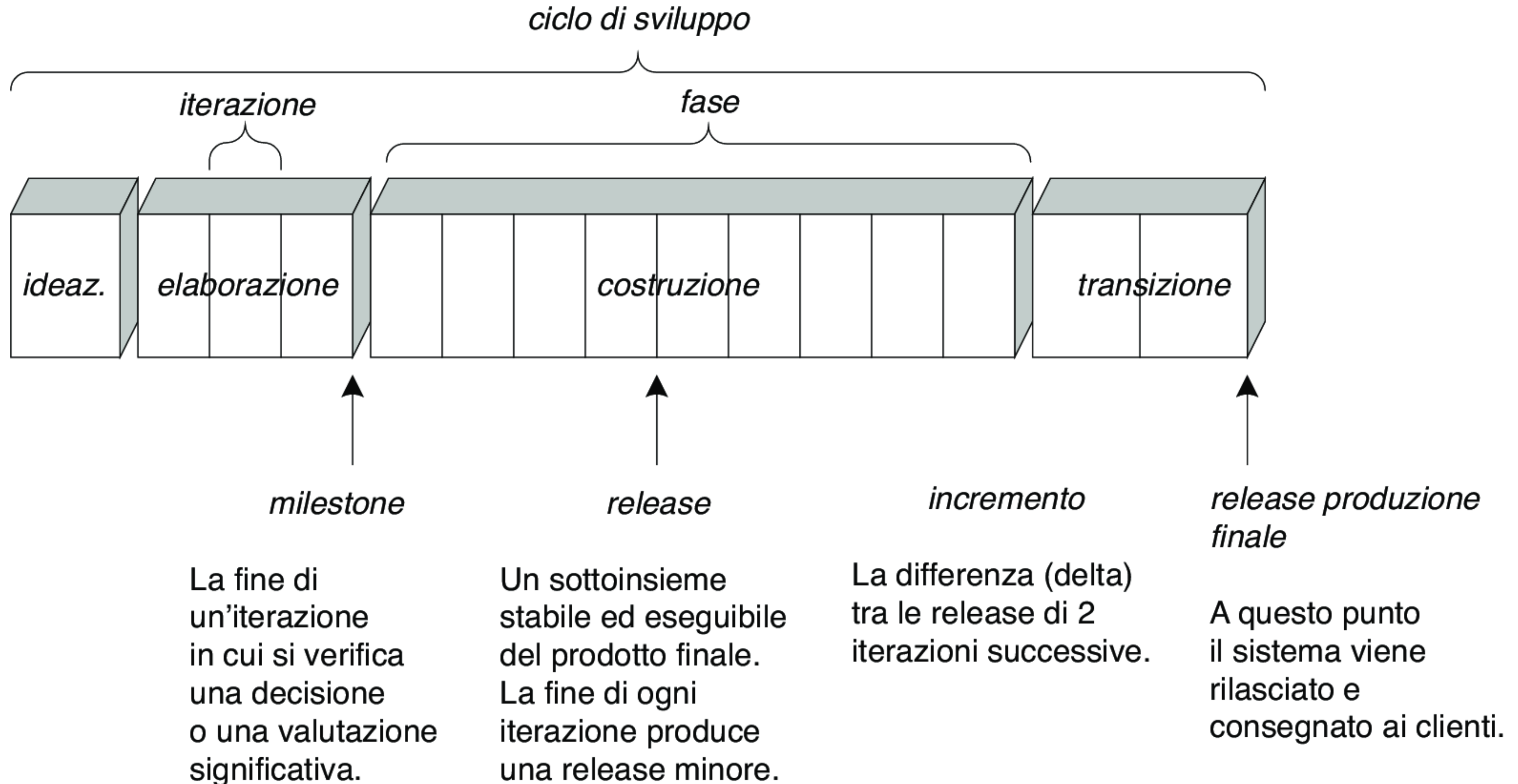
- Ogni iterazione genera una release
- Una release è un insieme di manufatti, previsti e approvati:
 - Fornisce una base approvata per le successive attività di analisi e sviluppo
- Un incremento è la differenza tra una release e quella successiva. Costituisce un passo in avanti verso il rilascio finale del sistema
 - Questo è il motivo per cui la UP si chiama "iterativo e incrementale"

Le fasi di UP

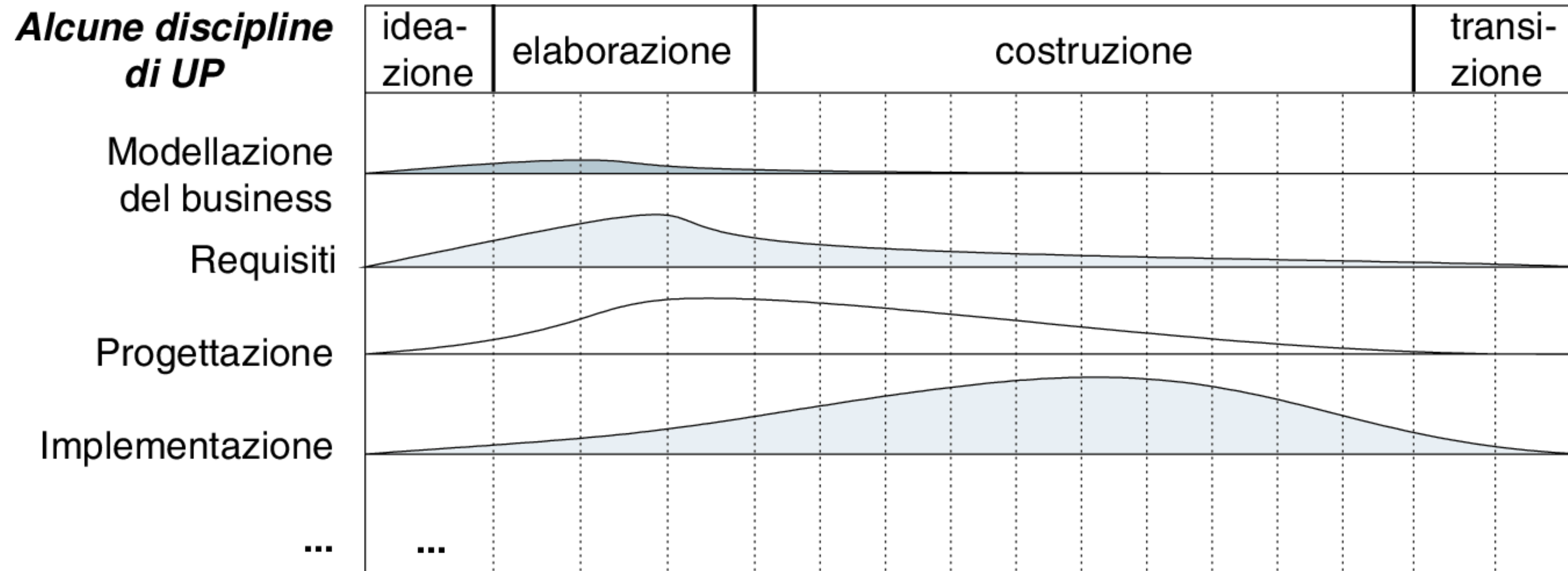
Un progetto UP organizza il lavoro e le interazioni in quattro fasi temporali:

- **Ideazione:** Avvio del progetto
 - *visione approssimativa, studio economico, portata, stime approssimazione dei costi e dei tempi*
- **Elaborazione:** Realizzazione del nucleo dell'architettura
 - *visione raffinata, implementazione iterativa del nucleo dell'architettura, risoluzione dei rischi maggiori e identificazione della maggior parte dei requisiti e stime più realistiche*
- **Costruzione:** Realizzazione delle capacità operative iniziali
 - *Implementazione iterativa degli elementi rimanenti, più facili e a rischio minore, e preparazione al rilascio*
- **Transizione:** Completamento del prodotto
 - *Beta tester, rilascio*

Struttura dell'UP



Fasi e discipline (o flussi di lavoro)



L'impegno relativo nelle discipline cambia a seconda delle fasi.

Questo esempio è solo un suggerimento, non è da prendere alla lettera.

- Questa figura è la chiave per comprendere UP!
- Per ogni fase si considererà:
 - L'attenzione in termini di flussi di lavoro
 - L'obiettivo per la fase
 - Il milestone alla fine della fase

Personalizzare il processo con lo scenario di sviluppo di UP

- Tra gli elaborati e le pratiche di UP, quasi tutto è opzionale
 - *Alcune pratiche e principi di UP sono fissi, come lo sviluppo iterativo e guidato dal rischio e la verifica continua della qualità*
 - *Tutte le attività e gli elaborati (modelli, diagrammi, documenti,...) sono opzionali con l'ovvia esclusione del codice*
- La scelta delle pratiche degli elaborati UP per un progetto può essere scritta in un breve documento chiamato scenario di sviluppo

Scenario di sviluppo

Disciplina	Pratica	Elaborato	Ideazione	Elaboraz	Costr	Transiz
		Iterazione	I1	E1..En	C1..Cn	T1..T2
Modellazione del business	modellazione agile workshop requisiti	Modello di Dominio		i		
Requisiti	workshop requisiti	Modello dei Casi d'Uso	i	r		
	esercizio sulla visione	Visione	i	r		
	votazione a punti	Specifica Supplementare	i	r		
		Glossario	i	r		
Progettazione	modellazione agile	Modello di Progetto		i	r	
	sviluppo guidato dai test	Documento dell'Architettura Software		i		
		Modello dei Dati		i	r	

i=Inizio, r=raffinamento

Non si è capito lo sviluppo iterativo o UP se...

- Si cerca di definire la maggior parte dei requisiti prima di iniziare la progettazione o l'implementazione
- Si cerca di finire la maggior parte del progetto prima di iniziare l'implementazione
- Si cerca definire in modo completo un'architettura e di affidarsi alla stessa prima della programmazione interattiva dei test
- Si pensa che la creazione di diagrammi UML e le attività di progettazione devono definire il progetto in dettaglio, e che la programmazione è semplicemente la traduzione meccanica di questi diagrammi
- Si pensa che ideazione = requisiti, elaborazione = progettazione e costruzione = implementazione
- Si ritiene che una durata adeguata per un'interazione sia tre mesi, anziché tre settimane
- Si pensa che adottare UP significhi svolgere quante più attività e produrre quanti più elaborati possibili
- Si prova a pianificare il progetto in dettaglio dall'inizio alla fine, in modo speculativo

Metodi e atteggiamenti agili

- Lo sviluppo agile è una forma di sviluppo iterativo che incoraggia l'agilità – ovvero una risposta rapida e flessibile ai cambiamenti
 - i metodi agili
 - applicano lo sviluppo iterativo – con iterazioni brevi e timeboxed – e la pianificazione iterativa
 - promuovono le consegne incrementali
 - sostengono valori “agili” – come semplicità, leggerezza, valore delle persone, comunicazione, ...
 - sostengono pratiche “agili” – come programmazione a coppie, TDD, refactoring, ...
- Qualsiasi processo iterativo, compreso UP, può essere applicato in uno spirito agile

Manifesto e principi agili

Manifesto per lo Sviluppo Agile di Software

Stiamo scoprendo modi migliori di creare software, sviluppandolo e aiutando gli altri a fare lo stesso. Grazie a questa attività siamo arrivati a considerare importanti:

gli individui e le interazioni	più che i processi e gli strumenti;
il software funzionante	più che la documentazione esaustiva;
la collaborazione con il cliente	più che la negoziazione dei contratti;
rispondere al cambiamento	più che seguire un piano.

Ovvero, fermo restando il valore delle voci a destra, consideriamo più importanti le voci a sinistra.

I principi sottostanti al Manifesto Agile

Seguiamo questi principi:

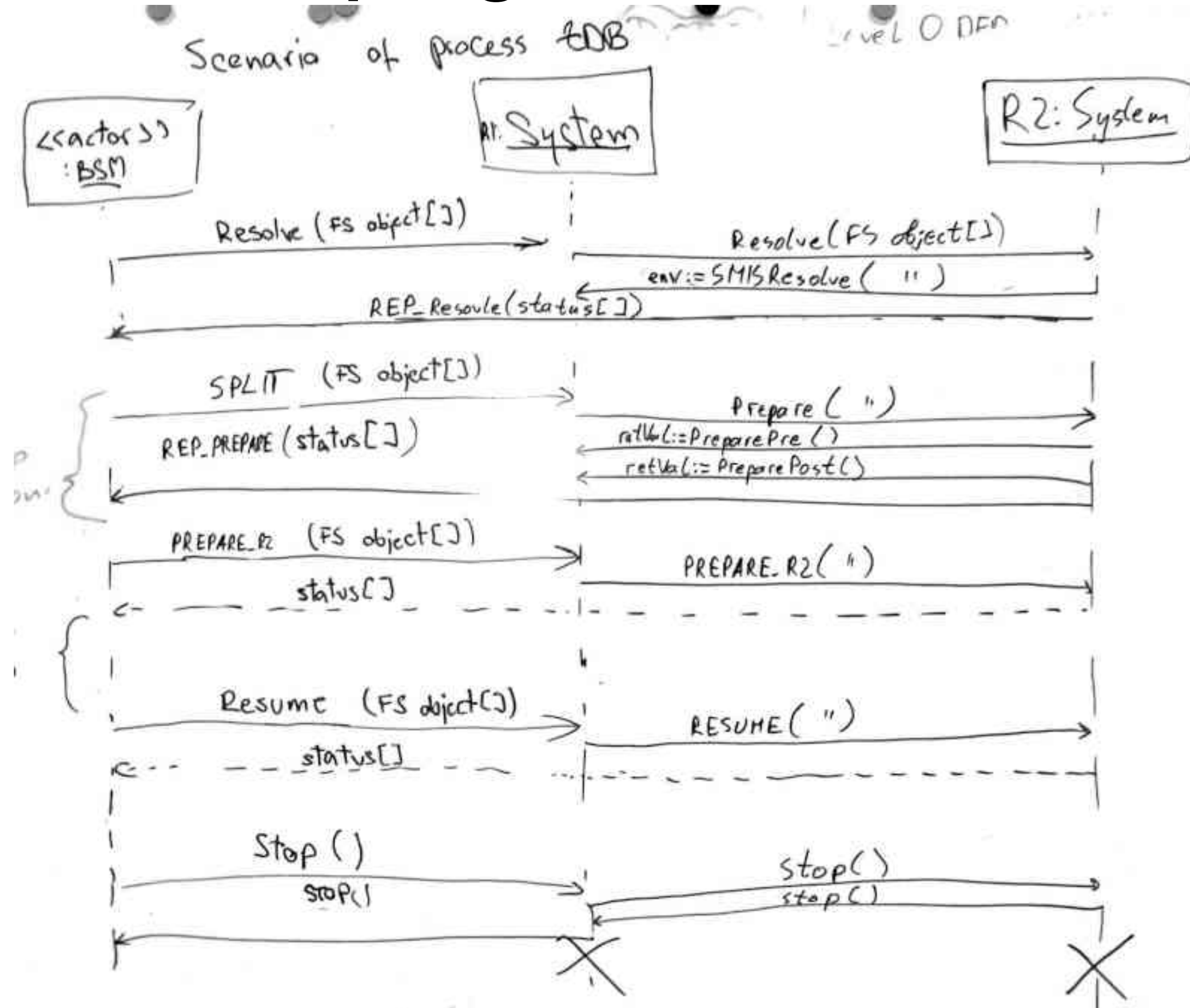
1. La nostra massima priorità è soddisfare il cliente rilasciando software di valore, fin da subito e in maniera continua.
2. Accogliamo i cambiamenti nei requisiti, anche a stadi avanzati dello sviluppo. I processi agili sfruttano il cambiamento a favore del vantaggio competitivo del cliente.
3. Consegniamo frequentemente software funzionante, con cadenza variabile da un paio di settimane a un paio di mesi, preferendo i periodi brevi.
4. Committenti e sviluppatori devono lavorare insieme quotidianamente per tutta la durata del progetto.
5. Fondiamo i progetti su individui motivati. Diamo loro l'ambiente e il supporto di cui hanno bisogno e confidiamo nella loro capacità di portare il lavoro a termine.
6. Una conversazione faccia a faccia è il modo più efficiente e più efficace per comunicare con il team ed all'interno del team.
7. Il software funzionante è il principale metro di misura di progresso.
8. I processi agili promuovono uno sviluppo sostenibile. Gli sponsor, gli sviluppatori e gli utenti dovrebbero essere in grado di mantenere indefinitamente un ritmo costante.
9. La continua attenzione all'eccellenza tecnica e alla buona progettazione esaltano l'agilità.
10. La semplicità – l'arte di massimizzare la quantità di lavoro non svolto – è essenziale.
11. Le architetture, i requisiti e la progettazione migliori emergono da team che si auto-organizzano.
12. A intervalli regolari il team riflette su come diventare più efficace, dopodiché regola e adatta il proprio comportamento di conseguenza.

Agile Modeling

Agile Modeling è basato su un insieme di pratiche e valori, tra cui

- adottare un metodo agile non significa evitare del tutto la modellazione
- lo scopo dei modelli e della modellazione è agevolare la comprensione e la comunicazione
- Modellare e applicare UML solo alle parti del progetto che sono difficili, insolite e insidiose
- usare lo strumento più semplice possibile
- La modellazione non va fatta da soli ma piuttosto a coppie (o in tre)
- tenere presente che tutti i modelli saranno incompleti e imprecisi

Un abbozzo UML di un diagramma di sequenza in un progetto reale



Cosa è UP agile

UP non è stato pensato per essere pesante o non agile

- molti elementi sono opzionali
- personalizzazione di UP

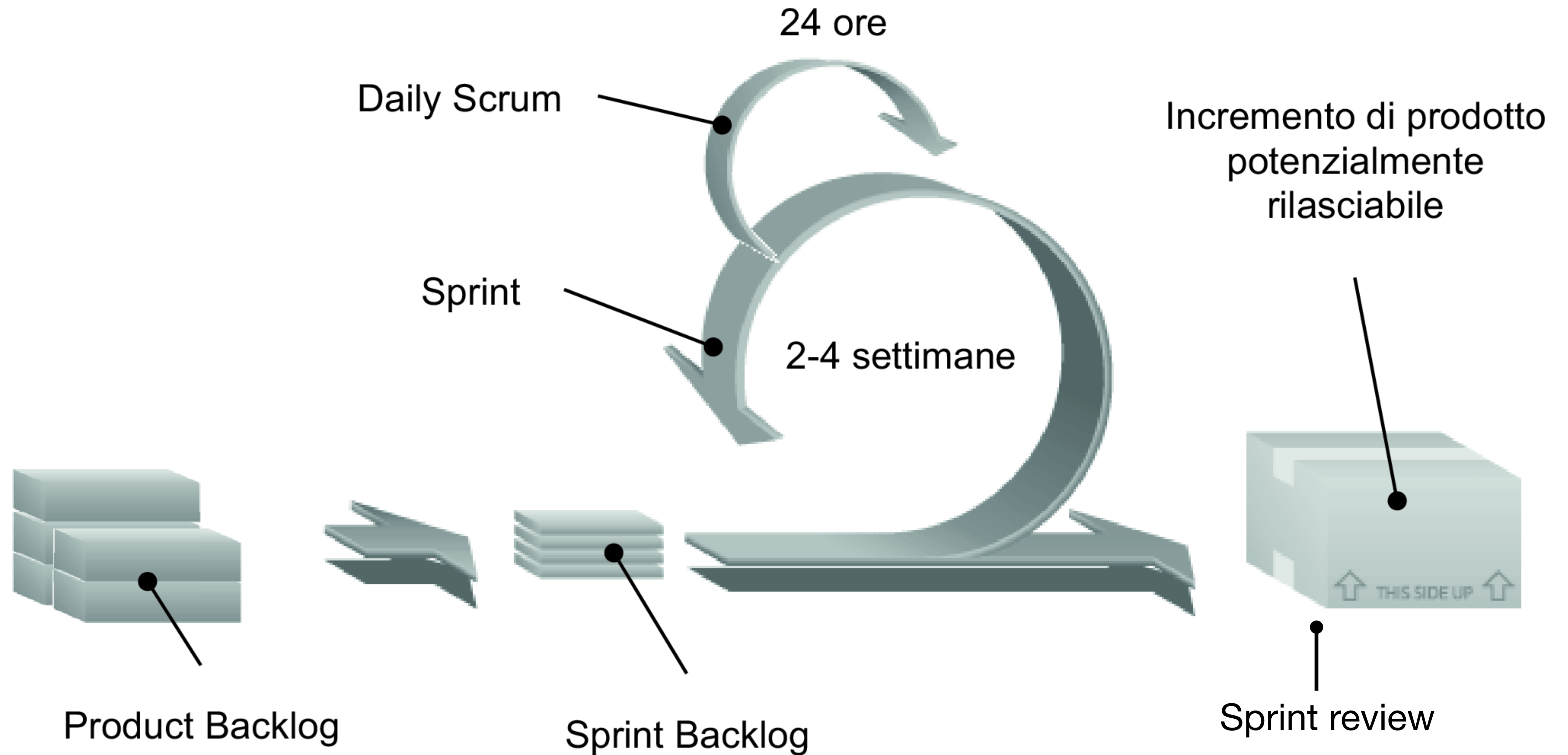
UP può essere applicato in modo agile (**UP agile**)

- un piccolo insieme di attività ed elaborati
- I requisiti e la progettazione non vengono completati prima dell'implementazione, ma emergono in modo adattivo durante una serie di interazioni, anche sulla base di feedback
- applicazione di UML secondo lo spirito della modellazione agile
- pianificazione iterativa e adattiva

Che cosa è Scrum

- Scrum è un metodo agile che consente di sviluppare e rilasciare prodotti software con il più alto valore per i clienti, nel più breve tempo possibile
- Scrum si occupa principalmente dell'organizzazione del lavoro e della gestione di progetti (può essere facilmente combinato con altri metodi)

Scrum



Termini Scrum

Termine Scrum	Definizione
Scrum	Un incontro giornaliero del team Scrum che esamina i progressi e definisce le priorità del lavoro da svolgere in quel giorno. Idealmente, questo dovrebbe essere un breve incontro faccia a faccia che includa l'intera squadra.
ScrumMaster	Lo ScrumMaster è responsabile di assicurare che il processo di Scrum sia seguito e guida il team nell'uso efficace di Scrum. È responsabile dell'interfacciamento con il resto dell'azienda e di assicurare che il team Scrum non venga deviato da interferenze esterne. Gli sviluppatori di Scrum sono convinti che lo ScrumMaster non debba essere considerato un project manager. Altri, tuttavia, potrebbero non sempre trovare facile vedere la differenza.
Sprint	Un'iterazione di sviluppo. Le sprint sono solitamente di 2-4 settimane.
Velocity	Una stima di quanto lavoro rimanente un team può fare in un singolo sprint. Capire la velocità di una squadra li aiuta a stimare ciò che può essere coperto in uno sprint e fornisce una base per misurare il miglioramento delle prestazioni.

Termini Scrum

Termine Scrum	Definizione
Team di sviluppo	Un gruppo auto-organizzatore di sviluppatori di software, che non dovrebbe superare le 7 persone. Sono responsabili dello sviluppo del software e di altri documenti essenziali del progetto.
Incremento potenzialmente rilasciabile	L'incremento software fornito da uno sprint. L'idea è che ciò dovrebbe essere "potenzialmente trasportabile", il che significa che si trova in uno stato finito e non è necessario alcun ulteriore lavoro, come il test, per incorporarlo nel prodotto finale. In pratica, ciò non è sempre realizzabile.
Product backlog	Questo è un elenco di elementi 'da fare' che il team Scrum deve affrontare. Possono essere definizioni di caratteristiche per il software, requisiti software, storie utente o descrizioni di compiti supplementari necessari, come la definizione dell'architettura o la documentazione utente.
Product owner	Un individuo (o eventualmente un piccolo gruppo) il cui compito è quello di identificare le caratteristiche o i requisiti del prodotto, dare la priorità a questi per lo sviluppo e rivedere continuamente la product backlog per garantire che il progetto continui a soddisfare le esigenze di business critiche. Il Product Owner può essere un cliente, ma potrebbe anche essere un product manager in una società di software o un rappresentante di altri stakeholder.

An Agile Game Developer

